

Semi-Supervised Learning

Semi-Supervised Learning is a type of Machine Learning where the model learns using:

- **Small amount of labeled data**
- **Large amount of unlabeled data**

It is a combination of:

- **Supervised Learning** → data has labels
 - **Unsupervised Learning** → data has no labels
-

Simple Definition

In Semi-Supervised Learning, only some data is tagged with correct answers, while most data is untagged.

The model uses both to improve learning.

Example

Suppose you want to build a **cat vs dog classifier**.

You have:

- 100 images with labels
 - Cat
 - Dog

and

- 10,000 images without labels.

Instead of labeling all 10,000 images manually, the model learns from the small labeled dataset and also uses the unlabeled data to improve accuracy.

Why We Use It

Labeling data is:

- Expensive
- Time-consuming
- Difficult sometimes

But unlabeled data is easily available.

So Semi-Supervised Learning helps reduce labeling work.

Working Process

Step 1: Train with labeled data

The model first learns from labeled examples.

Example:

Image	Label
--------------	--------------

Cat image	Cat
-----------	-----

Dog image	Dog
-----------	-----

Step 2: Predict unlabeled data

The model predicts labels for unlabeled data.

Example:

Image	Predicted Label
--------------	------------------------

Unknown image	Cat
---------------	-----

Step 3: Re-train the model

The confidently predicted data is added to training data.

The model becomes stronger with more data.

Types of Semi-Supervised Learning

1. Self-Training

- Model trains itself
- Predicts unlabeled data
- Uses high-confidence predictions again for training

Example

Teacher teaches some students, then students help teach others.

2. Co-Training

- Two models train each other
 - Each model teaches the other using different features
-

3. Graph-Based Methods

- Similar data points are connected in graph form
 - Nearby points likely belong to same class
-

Advantages

- Requires less labeled data
 - Reduces cost
 - Better accuracy than unsupervised learning
 - Useful when labeled data is limited
-

Disadvantages

- Wrong predictions may reduce accuracy
 - Training can become complex
 - Needs careful confidence checking
-

Reinforcement Learning (RL)

Reinforcement Learning is a type of Machine Learning where an **agent learns by interacting with the environment** and receives:

- **Reward** for correct action
- **Penalty** for wrong action

The main goal is to maximize the total reward.

Main Components of Reinforcement Learning

1. Agent

The learner or decision-maker.

Example:

- Robot
 - AI player
 - Self-driving car
-

2. Environment

The world where the agent works.

Example:

- Game board
 - Road
 - Robot area
-

3. Action

What the agent does.

Example:

- Move left
- Accelerate
- Pick object

4. State

Current situation of the environment.

Example:

- Position of car
- Game score
- Robot location

5. Reward

Feedback after action.

- Positive reward → good action
- Negative reward → bad action

Working Process

Step 1: Observe State

Agent checks the current situation.

Step 2: Take Action

Agent chooses an action.

Step 3: Receive Reward

Environment gives feedback.

Step 4: Learn

Agent updates knowledge and improves future decisions.

Types of Reinforcement Learning

1. Positive Reinforcement

Reward is given for correct action.

Example:

- Game points
 - Bonus score
-

2. Negative Reinforcement

Penalty or negative reward for wrong action.

Example:

- Losing health in a game
 - Traffic accident penalty
-

Important Terms

Policy

A strategy that tells the agent what action to take.

Value Function

Measures how good a state is.

Q-Value

Measures how good an action is in a particular state.

Popular RL Algorithms

1. Q-Learning

Learns the best action using Q-values.

2. SARSA

Learns using current action and next action.

3. Deep Q Network (DQN)

Uses Deep Learning with Q-Learning.

Advantages

- Learns automatically from experience
 - Can solve complex problems
 - Improves continuously
-

Disadvantages

- Needs large training time
 - High computational cost
 - Can make many mistakes during learning
-

Markov Decision Process (MDP)

Markov Decision Process (MDP) is a mathematical model used in Reinforcement Learning to make decisions step by step in an environment.

It helps an agent decide:

- Which action to take
 - In which state
 - To get maximum reward
-

Real-Life Example

Example: Robot Navigation

A robot wants to reach a destination.

- Current position = State
- Move left/right/up/down = Action
- Reaching goal = Reward
- Hitting obstacle = Penalty

The robot learns the best path.

Main Components of MDP

MDP mainly has 5 components:

$$MDP = (S, A, P, R, \gamma)$$

$$MDP = (S, A, P, R, \gamma)$$

1. State (S)

Represents the current situation.

Example

- Position of robot
- Current game level
- Traffic condition

2. Action (A)

Possible actions the agent can perform.

Example

- Move forward
 - Turn left
 - Stop
-

3. Probability (P)

Probability of moving from one state to another after an action.

Example

If robot moves right:

- 80% chance → correct move
 - 20% chance → slips
-

4. Reward (R)

Feedback received after taking action.

- Positive reward → good action
 - Negative reward → bad action
-

5. Discount Factor (γ)

Measures importance of future rewards.

$$0 \leq \gamma \leq 1$$

$$0 \leq \gamma \leq 1$$

- γ close to 1 → future rewards important
 - γ close to 0 → immediate rewards important
-

Markov Property

MDP follows the Markov Property:

Future state depends only on the current state and action, not on past history.

Working of MDP

Step 1: Observe current state

Agent checks current situation.

Step 2: Take action

Agent chooses an action.

Step 3: Move to next state

Environment changes state.

Step 4: Receive reward

Agent gets reward or penalty.

Advantages

- **Handles sequential decision making**
 - **Works well in uncertain environments**
 - **Foundation of Reinforcement Learning**
-

Disadvantages

- **Large state space becomes complex**
 - **Requires high computation**
 - **Difficult for real-world huge environments**
-

Bellman Equations

Bellman Equation is one of the most important concepts in Reinforcement Learning (RL) and Markov Decision Process (MDP).

It helps calculate:

- The value of a state
 - The best future reward
-

Simple Definition

Bellman Equation breaks a big problem into smaller subproblems.

It says:

The value of the current state = Immediate reward + Future rewards

Main Idea

The agent should not only think about:

- Current reward

but also about:

- Future rewards
-

Bellman Value Equation

$$V(s) = R(s) + \gamma \sum P(s' | s, a) V(s')$$

$$V(s) = R(s) + \gamma \sum P(s' | s, a) V(s')$$

Meaning of Symbols

Symbol Meaning

$V(s)$ Value of current state

$R(s)$ Immediate reward

γ Discount factor

Symbol Meaning

$(P(s' | s, a))$

$V(s')$ Future state value

Simple Meaning

Current value depends on:

1. Immediate reward
 2. Discounted future rewards
-

Discount Factor (γ)

$$0 \leq \gamma \leq 1$$

$$0 \leq \gamma \leq 1$$

- γ close to 1 $\rightarrow$ future rewards important
 - γ close to 0 $\rightarrow$ immediate rewards important
-

Example

Suppose:

- Current reward = 10
- Future value = 20
- $\gamma = 0.9$

Then:

$$V(s) = 10 + 0.9(20)$$

$$V(s) = 28$$

$$V(s) = 10 + 0.9(20)$$

Bellman Optimality Equation

Used to find the best action.

$$V^*(s) = \max_a [R(s, a) + \gamma \sum P(s' | s, a) V^*(s')]$$

$$V^*(s) = \max_a [R(s, a) + \gamma \sum P(s' | s, a) V^*(s')]$$

Meaning

The agent chooses the action that gives the maximum reward.

Q-Value Bellman Equation

Used in Q-Learning.

$$Q(s, a) = R(s, a) + \gamma \sum P(s' | s, a) \max_{a'} Q(s', a')$$

$$Q(s, a) = R(s, a) + \gamma \sum P(s' | s, a) \max_{a'} Q(s', a')$$

Why Bellman Equation is Important

It is used in:

- Q-Learning
- Value Iteration
- Policy Iteration
- Deep Reinforcement Learning

It is the foundation of RL algorithms.

Advantages

- Finds optimal solutions
 - Handles sequential decisions
 - Works with uncertain environments
-

Disadvantages

- Computationally expensive

- **Difficult for very large state spaces**

Policy Evaluation using Monte Carlo

Policy Evaluation using Monte Carlo is a Reinforcement Learning method used to find:

- How good a policy is
- The value of each state

It learns by using complete episodes and averaging rewards.

Main Idea

The agent:

- Starts from a state
- Follows a policy
- Reaches the terminal state
- Observes total reward

Then it updates the state value.

What is a Policy?

A policy tells the agent what action to take.

$$\pi(s) = a$$

Meaning:

- In state s
- Take action a

$$\pi(s) = a$$

Monte Carlo Concept

Monte Carlo methods learn from:

- Experience
- Random sampling
- Complete episodes

They do not require knowledge of environment probabilities.

State Value Function

The value of a state is:

$$V(s) = \mathbb{E}[G_t \mid S_t = s]$$

$$V(s) = \mathbb{E}[G_t \mid S_t = s]$$

Meaning of Symbols

Symbol Meaning

$V(s)$ Value of state

G_t Total return

$\mathbb{E}$ Expected average reward

Return in Monte Carlo

Return means total future reward.

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

Working Process

Step 1: Initialize Policy

Choose a policy.

Example:

- Always move right
-

Step 2: Generate Episode

Run full episode from start to end.

Example:

- Start game
 - Reach goal
 - Episode ends
-

Step 3: Calculate Return

Compute total rewards received.

Step 4: Update State Value

Average all returns of that state.

Formula for State Value Update

$$V(s) = \frac{\text{Sum of returns}}{\text{Number of visits}}$$

$$V(s) = \frac{\text{Sum of returns}}{\text{Number of visits}}$$

Types of Monte Carlo Methods

1. First-Visit Monte Carlo

Updates state value only on the first visit in an episode.

2. Every-Visit Monte Carlo

Updates value every time the state appears.

Example

Suppose state A gets returns:

- Episode 1 → 10
- Episode 2 → 20

- Episode 3 → 15

Then:

$$V(A) = \frac{10 + 20 + 15}{3}$$

$$V(A) = 15$$

$$V(A) = \frac{10 + 20 + 15}{3} = 15$$

Advantages

- Simple to understand
 - No environment model needed
 - Works well for episodic tasks
-

Disadvantages

- Must wait until episode ends
- Slow learning
- High variance in rewards

Policy Iteration

Policy Iteration is a Reinforcement Learning algorithm used to find the optimal policy in a Markov Decision Process (MDP).

It works by repeating two steps:

1. Policy Evaluation
2. Policy Improvement

until the best policy is found.

Simple Definition

Policy Iteration is a method where the agent:

- Evaluates the current policy
 - Improves the policy
 - Repeats the process until the optimal solution is reached
-

Main Idea

The algorithm continuously asks:

- “How good is my current policy?”
- “Can I improve it?”

If yes → improve it again.

Important Terms

Policy

A strategy that tells which action to take.

$$\pi(s) = a$$

Meaning:

- In state s
- Take action a

$$\pi(s) = a$$

State Value Function

Measures how good a state is under a policy.

$$V^{\pi}(s)$$

$$V^{\pi}(s)$$

Steps of Policy Iteration

Step 1: Initialize Policy

Start with a random policy.

Example:

- Always move right
-

Step 2: Policy Evaluation

Calculate state values using the current policy.

Bellman Equation:

$$V^{\pi}(s) = R(s) + \gamma \sum P(s' | s, a) V^{\pi}(s')$$

$$V^{\pi}(s) = R(s) + \gamma \sum P(s' | s, a) V^{\pi}(s')$$

Step 3: Policy Improvement

Improve the policy by choosing better actions.

$$\pi'(s) = \arg \max_a [R(s, a) + \gamma \sum P(s' | s, a) V^{\pi}(s')]$$

$$\pi'(s) = \arg \max_a [R(s, a) + \gamma \sum P(s' | s, a) V^{\pi}(s')]$$

Step 4: Repeat

Repeat:

- **Policy Evaluation**
- **Policy Improvement**

until policy stops changing.

That final policy is the optimal policy.

Flow of Policy Iteration

Random Policy

↓

Policy Evaluation

↓

Policy Improvement

↓

Better Policy

↓

Repeat Until Optimal

Example

Imagine a robot finding the shortest path.

Initial Policy

Robot moves randomly.

Evaluation

Robot learns which paths give better rewards.

Improvement

Robot changes actions toward better paths.

Final Result

Robot learns the best route.

Why Policy Iteration Works

Each improvement step makes the policy:

- **Better or equal**
- **Never worse**

Eventually, the best policy is reached.

Advantages

- **Finds optimal policy**
 - **Guaranteed improvement**
 - **Efficient for small environments**
-

Disadvantages

- **Computationally expensive**
- **Difficult for huge state spaces**
- **Needs repeated calculations**

Value Iteration

Value Iteration is a Reinforcement Learning algorithm used to find the optimal value function and optimal policy in a Markov Decision Process (MDP).

It repeatedly updates state values until the best values are found.

Main Idea

The algorithm asks:

“What is the best possible future reward from this state?”

Then it updates the value again and again.

Goal of Value Iteration

To find:

- Optimal state values
 - Optimal policy
-

Bellman Optimality Equation

Value Iteration uses this equation:

$$V^*(s) = \max_a [R(s, a) + \gamma \sum P(s' | s, a) V^*(s')]$$

$$V^*(s) = \max_a [R(s, a) + \gamma \sum P(s' | s, a) V^*(s')]$$

Meaning of Symbols

Symbol Meaning

$V^*(s)$ Optimal value of state

$R(s, a)$ Immediate reward

γ Discount factor

$(P(s' | s, a))$

Symbol Meaning

$\max_a$ Choose best action

Working Process

Step 1: Initialize Values

Start with random values.

Example:

$$V(s) = 0$$

$$V(s) = 0$$

Step 2: Update Values

Use Bellman equation repeatedly.

For every state:

- Check all possible actions
 - Calculate future rewards
 - Choose maximum reward
-

Step 3: Repeat

Continue updating until values stop changing.

This is called convergence.

Step 4: Extract Optimal Policy

Choose the best action from final values.

$$\pi^*(s) = \arg \max_a [R(s, a) + \gamma \sum P(s' | s, a) V^*(s')]$$

$$\pi^*(s) = \arg \max_a [R(s, a) + \gamma \sum P(s' | s, a) V^*(s')]$$

Example

Imagine a robot finding the shortest path.

- Each cell has rewards
- Robot checks all directions
- Updates path values repeatedly
- Finally learns best route

Flow of Value Iteration

Initialize Values

↓

Apply Bellman Equation

↓

Update State Values

↓

Repeat Until Stable

↓

Generate Optimal Policy

Difference Between Policy Iteration and Value Iteration

	Feature Policy Iteration	Value Iteration
Focus	Policy + Value	Value only first
Steps	Two separate steps	Single update step
Speed	Sometimes faster	Simpler method

Advantages

- Simple and powerful
 - Finds optimal policy
 - Widely used in RL
-

Disadvantages

- **Computationally expensive**
 - **Slow for very large state spaces**
-

Q-Learning

Q-Learning is a popular **Reinforcement Learning (RL)** algorithm used to find the best action for an agent in a given state.

It learns by:

- Trying actions
 - Receiving rewards
 - Improving decisions over time
-

Simple Definition

Q-Learning is a model-free Reinforcement Learning algorithm that learns the optimal action-value function using rewards and experience.

Main Idea

The agent learns:

“Which action gives the highest reward in a particular state?”

What is Q-Value?

Q-value means:

- Quality of an action

It measures how good an action is in a particular state.

$$Q(s, a)$$

Meaning:

- State = s
- Action = a

$$Q(s, a)$$

Goal of Q-Learning

Find the optimal Q-values so the agent can choose the best action.

Q-Learning Formula

$$Q(s, a) = Q(s, a) + \alpha \left[R + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

$$Q(s, a) = Q(s, a) + \alpha [R + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Meaning of Symbols

Symbol	Meaning
--------	---------

$Q(s, a)$	Current Q-value
-----------	-----------------

α	Learning rate
----------	---------------

R	Reward
-----	--------

γ	Discount factor
----------	-----------------

$\max_{a'} Q(s', a')$	Maximum future reward
-----------------------	-----------------------

s'	Next state
------	------------

Simple Meaning of Formula

New Q-value =

- Old Q-value
 - - Learning from new experience
-

Working Process

Step 1: Observe State

Agent checks current state.

Example:

- Robot location

Step 2: Choose Action

Agent selects an action.

Example:

- Move left/right

Step 3: Receive Reward

Environment gives reward or penalty.

Step 4: Update Q-Value

Use Q-Learning formula.

Step 5: Repeat

Continue until agent learns optimal behavior.

Q-Table

Q-Learning often uses a table called **Q-Table**.

State Left Right

A 2 8

B 5 1

The agent chooses the action with highest Q-value.

Example:

- In state A → choose Right

Exploration vs Exploitation

1. Exploration

Trying new actions.

2. Exploitation

Using known best action.

A balance between both is important.

Example

Imagine a mouse finding cheese in a maze.

- Wrong path → penalty
- Correct path → reward

After many attempts, the mouse learns the best route.

That is Q-Learning.

Advantages

- Simple and effective
 - No environment model required
 - Learns optimal policy
-

Disadvantages

- Slow for huge state spaces
 - Needs many training episodes
 - Q-table becomes very large
-

SARSA (State-Action-Reward-State-Action)

SARSA is a **Reinforcement Learning (RL)** algorithm used to learn the best policy by interacting with the environment.

SARSA stands for:

(State, Action, Reward, Next State, Next Action)

Main Idea

SARSA learns from:

1. Current state
2. Current action
3. Reward received
4. Next state
5. Next action chosen

That is why it is called SARSA.

What is Q-Value?

Q-value measures how good an action is in a state.

$$Q(s, a)$$

$$Q(s, a)$$

SARSA Update Formula

$$Q(s, a) = Q(s, a) + \alpha[R + \gamma Q(s', a') - Q(s, a)]$$

$$Q(s, a) = Q(s, a) + \alpha[R + \gamma Q(s', a') - Q(s, a)]$$

Meaning of Symbols

Symbol Meaning

$Q(s, a)$ Current Q-value

α Learning rate

R Reward

γ Discount factor

$Q(s', a')$ Next state-action value

s' Next state

a' Next action

Simple Meaning of Formula

New Q-value depends on:

- Current knowledge
 - Reward received
 - Next action actually taken
-

Working Process of SARSA

Step 1: Observe Current State

Agent checks current state s .

Step 2: Choose Action

Choose action a using current policy.

Step 3: Perform Action

Agent performs action.

Step 4: Receive Reward and Next State

Environment gives:

- Reward R
 - Next state s'
-

Step 5: Choose Next Action

Choose next action a' from new state.

Step 6: Update Q-Value

Apply SARSA formula.

Repeat Process

Continue learning until optimal policy is found.

Why SARSA is Called On-Policy

SARSA learns using the **same policy** it follows.

It updates values based on the action actually selected.

Example

Imagine a robot learning to walk safely.

- Safe path $\rightarrow$ reward
- Dangerous path $\rightarrow$ penalty

SARSA learns cautiously because it follows its current behavior policy.

SARSA Flow

Current State (s)

↓

Choose Action (a)

↓

Take Action

↓
Receive Reward (R)
↓
Next State (s')
↓
Choose Next Action (a')
↓
Update Q-Value

Difference Between SARSA and Q-Learning

Feature	SARSA	Q-Learning
Type	On-policy	Off-policy
Update Uses	Actual next action	Maximum possible action
Learning Style	Safer	More aggressive

Example Difference

Q-Learning

Chooses the best possible future action even if risky.

SARSA

Chooses the action actually followed by policy, usually safer.

Advantages

- Stable learning
 - Safer in risky environments
 - Works well in real-world tasks
-

Disadvantages

- Slower learning sometimes
- May not always find fastest optimal path
